

Kalshiman

A bot I built so I could stop working and stay home with my family.

The long version. Written by hand, mostly at 2 AM,
and honest about the parts that did not work.

For the investor, the student, the skeptic, and anyone who has ever
wondered what they would do if money stopped being the thing they think about all day.

Last updated 2026-08-20 07:58 UTC

What's inside

A note before the numbers – the why, and the dream stated plainly

1. The 92-cent question – the edge, and the catch
2. Where I was wrong – the confessions, with receipts
3. Where I was right – the ideas that held up
4. The rebuild – 105,000 rounds, one model, no cheating
5. How it works – the three layers and the exact rules
6. The receipts – including the part where the backtest told me I was losing
7. We're not done – the engine as it stands (new)
8. Risk – the stop policy, and the hatch I had to tune
9. The economics – one bot, a fleet, a thousand bots
10. The pool – how a fleet of bots works together
11. Where I'm going next
12. Straight answers

One last thing – if you're smart and you read all of this, yours

Everything in this document is paper. The real account is untouched, and it stays that way until the record earns it.

A note before the numbers

I want to be honest about why this exists, because the why is half the story. I want to stay home with my family. Not work-from-home - home. I want to be there for breakfast, for the school run, for the boring Tuesday afternoons that are actually the whole point. And I have come to believe that no one is ever going to pay me more than me. The people who decide my salary are not as motivated as I am. They have other budgets. I have one life.

Everyone should be able to just exist. That is not a slogan I picked up somewhere - it is the sentence I kept writing in notebooks while I was building this thing. The world is getting more expensive and stranger by the day, and the old deal - trade your hours for money until you are too tired to enjoy the money - is getting worse on both ends. So I started building the only thing I could think of that trades my hours for me, in markets where the answer is usually known before the round is over.

This has taken weeks of nights. I stopped counting the 2 AM sessions, but I did count the restarts - the log folder has headless-restart1 through headless-restart86, and I touched the config five times in one day once because something kept rewriting it behind my back. I pulled 105,888 finished rounds and built the dataset by hand, field by field. I trained a model in pure JavaScript because I wanted the whole thing to run on one machine, in the corner of my house, forever. I rebuilt the strategy from scratch after it lost real money, and then I rebuilt the backtest after I caught the backtest lying to me.

Here is the honest version of the dream, before the numbers dress it up. A family of four in the US can live on about \$1,800 a month. One bot, at thirty trades a day, projects more than that - eventually, at stakes the balance can back. A college student covering food and fun needs about ten trades a day. A group of friends running ten bots is making hobby money while they sleep. A hundred is a quit-your-job number. A thousand is the dream - and the caveats are printed right next to it, because I have been wrong enough to know that every dream needs a margin note.

I asked you once, on the onboarding screen: one grain of rice that doubles every day, or a hundred grains a day?
This system is the first option. It looks like nothing for a while, and then it stops looking like nothing.

The title of this document used to be a technical breakdown. This version is closer to what it actually is: an explanation of my madness, and a call to action. The madness is that I refuse to accept that 'work until you die' is the only setting. The call to action is that this works better with more of us - more bots, more markets, more people running them - and the last chapter tells you exactly how the referral side of that works.

1. The 92-cent question

Walk up to a gold market fifteen minutes before it closes. The price is 92 cents for 'up.' Ask a stranger what happens and they will shrug. Ask the market itself, and it has already told you: the buyers have decided, and almost all of the time – on my measured record, the favored side settles as predicted – that side is right.

So where's the catch? The catch is that 97% isn't 100%. The other 3% of the time, the 92-cent side collapses, and a loss costs you 92 cents – almost eleven times what a win pays. That's the whole game in one paragraph: win small, often. Lose small, rarely. Never let the two get out of balance. I understood the first half on day one. The second half took weeks, a real-money loss, and a backtest that made me sit down.

The measured convergence, from 105,000 finished rounds in my dataset (the favored side settles as predicted):

Market	Rounds	Resolved as favored
Silver hourly	8,225	98.5%
Gold 15m	7,563	97.1%
Crypto 15m (BTC/ETH/SOL/XRP/DOGE/ZEC)	~24,000	98-99.4%
Oil 15m (WTI)	1,580	99.2%

~~Hourly gold and silver are the same edge, just longer — sure thing, what could go wrong?~~

(pencil) This is where I was wrong. The hourly markets resolve the wrong way a third of the time, and they became the entire loss. The 15-minute window is the edge. The hourly window is a coin flip wearing a suit.

My first bot got the first half of the game right and the second half catastrophically wrong. It won 79% of the time and lost money anyway, because its losses averaged five times its wins. I also learned that trading fails in exactly two directions. Too hot: trust every hunch, chase every wiggle, get stopped out on noise. Too cold: wait for certainty that never comes, or let a formula stop you out of winners because it can't tell a dip from a disaster.

The fix turned out to be a balance, not a formula: three layers, each one catching the other two's failure mode. A statistician who is rarely wrong, an observer who reads the room, and a disciplined hand. Chapter five is about them. Chapters two and three are about what I had to be wrong about first.

2. Where I was wrong – the confessions, with receipts

Before the rebuild, my bot ran on real money for a few weeks. The record is not pretty, and it is the reason everything after it exists. I keep this table where I can see it, because every row taught me something I still use.

What I measured	Value
Settled trades	291
Win rate	78.7%
Net P&L	-\$77.85
Worst drawdown	-\$91.43
Average win / average loss	+12.3c / -48.5c
Median loss	-100.7c (a full reversal)

The math is one line: with 12-cent wins and 48-cent losses, break-even needs a 134% win rate. My bot was winning at 79% and losing anyway. Here is what I was wrong about, in the order I learned it:

- Wrong: win rate is the score. It is not. The score is win size against loss size. A 95% win rate with 10-cent wins and 90-cent losses is a donation machine.
- Wrong: hourly gold and silver are 'sure things.' They resolve the wrong way 27-36% of the time on my dataset. On the faithful replay they lost \$75 of the \$76 total loss. They were the trap wearing the edge's clothes.
- Wrong: \$200 becomes \$500 in month two. I said that in an earlier draft and then I did the math properly. Compounding is real, but it is percent-of-position, not magic. The corrected table is in chapter nine, and it is still good – just honest.
- Wrong: hold longer means make more. Sometimes it does – the reviews showed we left 2 cents on the table. But at scale, holding everything turns winners into losers. Banking the wins was the answer.
- Wrong: the 25-cent hatch is fine. The sweep said otherwise: 15 cents is the knee. At 25, the final-minute hold rode too many 15-24-cent dips to full losses. At 15, the hatch catches them at the stop floor. Below 12, it starts eating recoveries.
- Wrong: the backtest was fine. It was not. My first backtest entered at a close-minus-3-minute snapshot, skipped the GBM gate, let stops never fire, and priced the EV as if every near-certain converged. It said crypto was a gold mine. The faithful replay – the live engine's own functions on real candle books – said the truth: the current config lost \$76. Chapter seven is what it found.
- Wrong: I can trust a hunch about oil. WTI 15m whipsawed through every early entry. It is banned now, and the ban has a number next to it. XRP and exact-spot crypto markets are banned too: they are not the game.

3. Where I was right – the ideas that held up

It would be easy to read chapter two and think nothing survived. That is not true either. Some of what I believed from the start held up under the worst test I could build, and I want to give those ideas their credit, because they are the ones worth scaling.

- Right: the 92-cent idea. Buy the near-certain side late, bank the small win, cap the rare wrong call. The full-history replay, with the current lanes, nets positive on gold 15m – 563 trades, 77% win rate, and it is the most consistent lane in the entire dataset.

- Right: bank the wins. 452 of the 608 shipped-config trades are bow-outs – small locked gains. That is the profit engine. The instinct that felt cowardly – take the win instead of riding – is the entire business model.

- Right: the escape hatch. The idea was correct: a last-ditch exit in the final minute so a collapse cannot run the whole way. I had the width wrong, not the concept. Once tuned to 15 cents, it saved \$12.92 on 37 trades that would have lost \$18.94 held to resolution.

- Right: cut the lanes that bleed. The day I said 'we lost three golds in a row, we are not acting fast enough,' the instinct was to tighten. The data says the tighter version was right – the lane cut flipped the whole system from -\$76 to +\$3.52 before the hatch tuning even started.

- Right: keep a journal. Every close now gets a written post-mortem: what happened, what holding would have made, and a verdict. The model got smarter by reading its own mistakes, not by adding parameters.

- Right: demand an honest backtest. You insisted the backtest could not deviate from the live system, or we would never know if it truly works. That insistence is what caught the -\$76. It is the single most valuable review this project ever got.

- Right: the pool. One bot can watch a handful of markets. A daisy chain of workers sharing verified information can watch all of them. The economics of that idea – chapter ten – are why this document ends the way it does.

(pencil) The pattern: every time I was wrong, I was trading a market I did not measure. Every time I was right, I was trusting a number I had measured. The lesson is embarrassingly simple.

4. The rebuild – data before opinions

I stopped guessing and started measuring. I pulled every finished round I could find – 105,888 of them, across 16 series – and laid them out the way a trader sees them: what the market looked like before the entry, how it moved afterward, and where it landed.

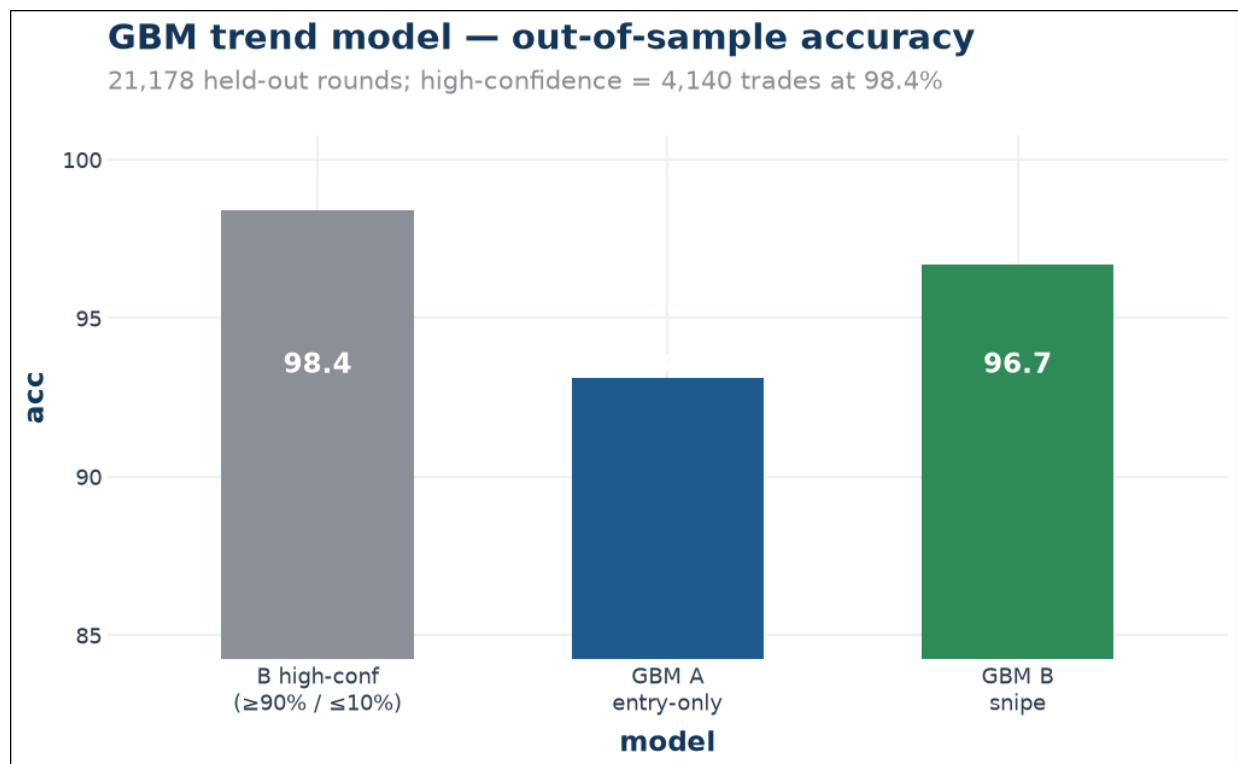
Every row in the dataset is structured the same way:

- entryPoint: the snapshot at close-minus-3-minutes – implied price, opening implied, deltas, spread, volume rate, open-interest change, range, hour, and the last three outcomes of the series.
- previewCandles: the last eight one-minute candles before the decision point.
- trendPath: the candle path after decision to close – the suspected trend the model learns to read.
- resolution and fills: where it landed, plus the account's real trades on that round.

Then I audited the data for leaks, because a model that reads the answer before it answers isn't a model, it's a cheat sheet. The audit passed: no future candles, no same-closeTime parallel-strike outcomes leaking into history features, no suspicious train/test correlations, and a strictly chronological split.

Out of that dataset I trained a gradient-boosted model – 200 trees, depth three, built in pure JavaScript with no external ML dependencies, so the whole thing runs on one machine. On rounds it had never seen:

Model	What it sees	Accuracy (out-of-sample)
Entry-only	the snapshot before you enter	93.1%
Snipe	plus the first two minutes of the trend	96.7%
Snipe, high-confidence	its strongest calls only	98.4% (4,140 trades)



The model's out-of-sample accuracy, strongest to strongest

I also built the honest evaluation loop: a dashboard that plays every round back, shows the model's reasoning, and scores it against reality. The dashboard only evaluates rounds the model never trained on, so what you see is what the model would actually do - not a backtest with the answers visible.

And then I built the honest backtest on top of the honest dataset - the one that reuses the live engine's actual functions, walks real candle books minute by minute, and tells the truth even when the truth is ugly. Chapter seven is that truth, in full.

5. How it works now – three layers and the exact rules

Watch a good trader for ten minutes and you will notice they don't compute, they observe – the book, the candles, the way the market is moving – and they ask one question: does this look like the setups that win? My system is three people in one.

A statistician. The GBM model. It has read 105,000 finished rounds and knows, to the percentage point, how often a 92-cent gold side actually settles. When it says 98%, it is right 98.4% of the time. It is calibrated, and it is never surprised.

A quiet observer. A small language model (Qwen3-1.7B, running locally via Ollama) reads each round the way a person would – the book, the path, the history – and gives a one-line opinion: yes, no, or pass. It is often wrong on its own, which is exactly why it is never in charge. Its opinions are written down and scored later, like a trader's journal.

A disciplined hand. It only acts when the statistician is confident and the observer agrees. Once in, it does the hard part: it lets a dip breathe, and if it does leave, it leaves at the stop level – never at whatever panicked price the market printed while it wasn't looking.

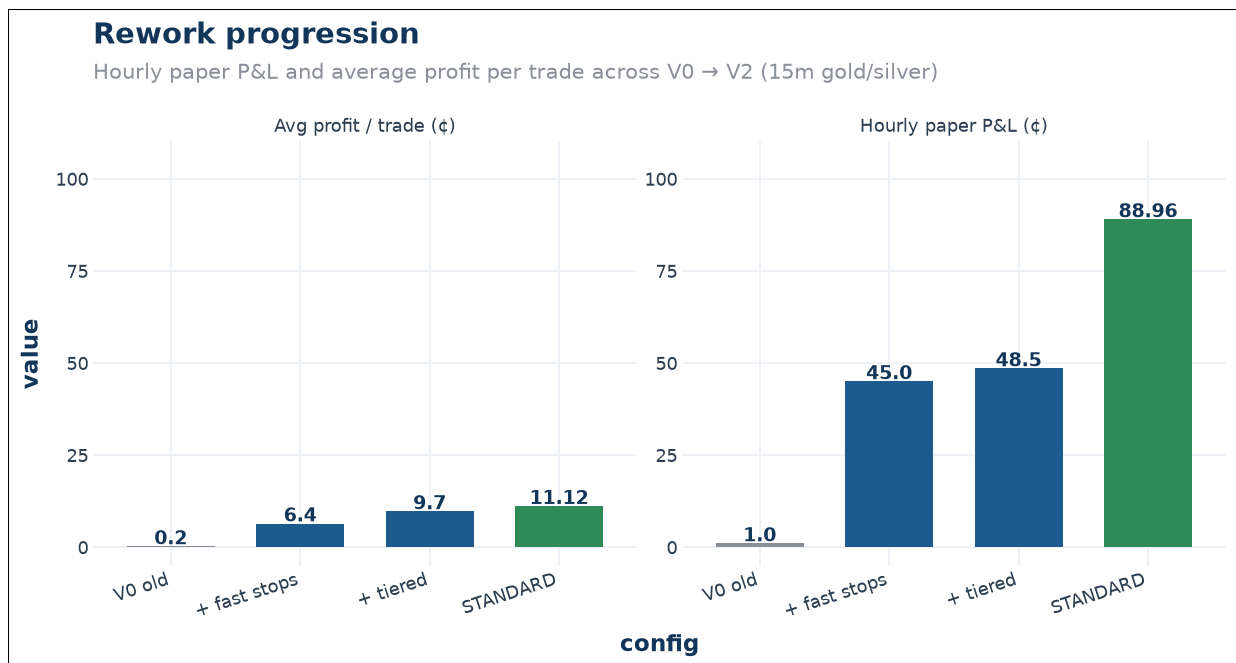
Step	What the code actually does
Scan	every 5s: gold15-late + universal snipe (1.5-8m to close); hourly gold/silver frozen
Score	researcher pulls 1m candles; logistic + GBM + GBM-snipe probabilities
Gate	price band, timing, EV, liquidity - then the GBM bar: $\geq 85\%$ or $\leq 15\%$ and agreement
Vetoes	gbm-low-conf - gbm-disagree - gbm-pending - gbm-warmup - gbm-timing (<4m to close below 95c)
Enter	paper trade, \$1
Manage	every tick + a 1.5s fast stop loop (paper AND live)
Win	bank at 99-99.5c or hold to resolution
Loss	tiered stop 15c ($\geq 90c$ entries) / 8c; 3-minute grace; limit-stop fill at the stop level
Final 2 min	GBM-confirmed trades hold through noise, with a 15c emergency quit in the final minute
After	written post-mortem: result, heldPnl, verdict - feeds retraining

Every rule in that table exists because a specific losing pattern taught it to me. The three-minute grace exists because nine of fifteen stops used to recover and win. The stop-level fill exists because a 15-cent stop once booked a 50-cent loss. The entry-timing guard exists because a trade entered 3.6 minutes before the bell has no time to survive a dip and recover. The 15-cent hatch exists because I swept it from 25 down to 5 and watched the curve tell me where the knee was. This is a system that keeps its receipts.

6. The receipts – including the part where the backtest told me I was losing

Here is the rework, hour by hour, on paper. Each row is the change that made the next row possible.

What changed	Trades	Result	Avg/trade
The old system	5	+1c, one loss	+0.2c
Paper stops that actually work	7	+45c, one loss	+6.4c
Stops that respect the entry	5	+48.5c, no losses	+9.7c
Stops that wait out noise	8	+88.96c, no losses	+11.12c

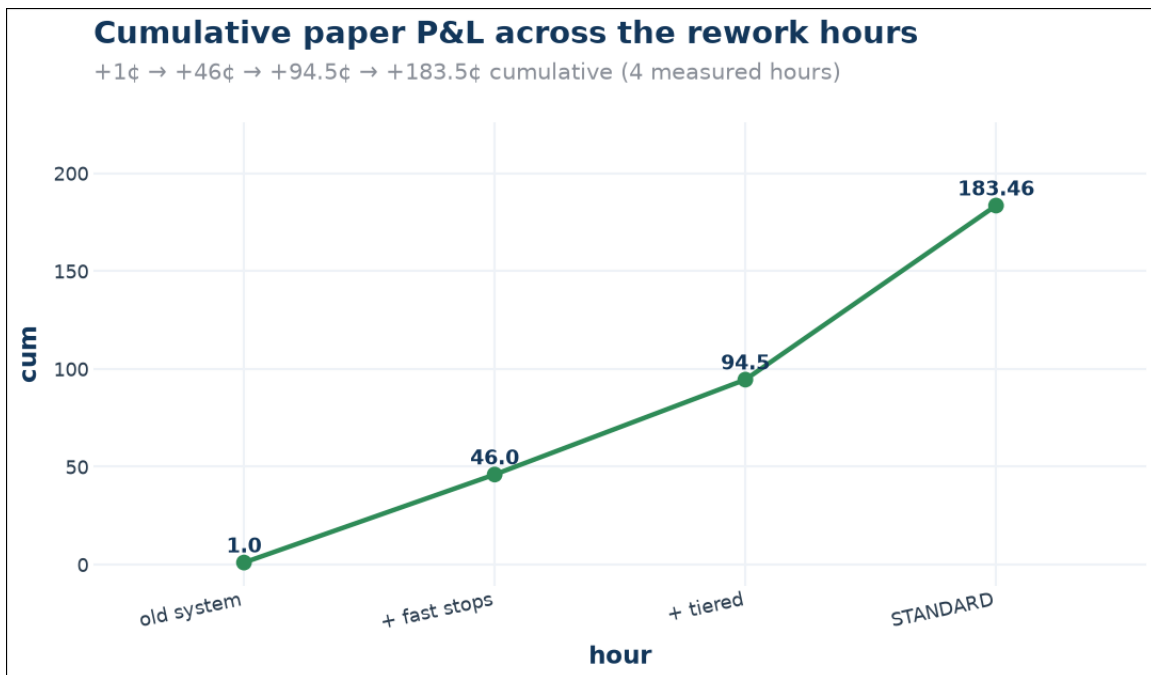


The same story as a chart: money per hour, and money per trade

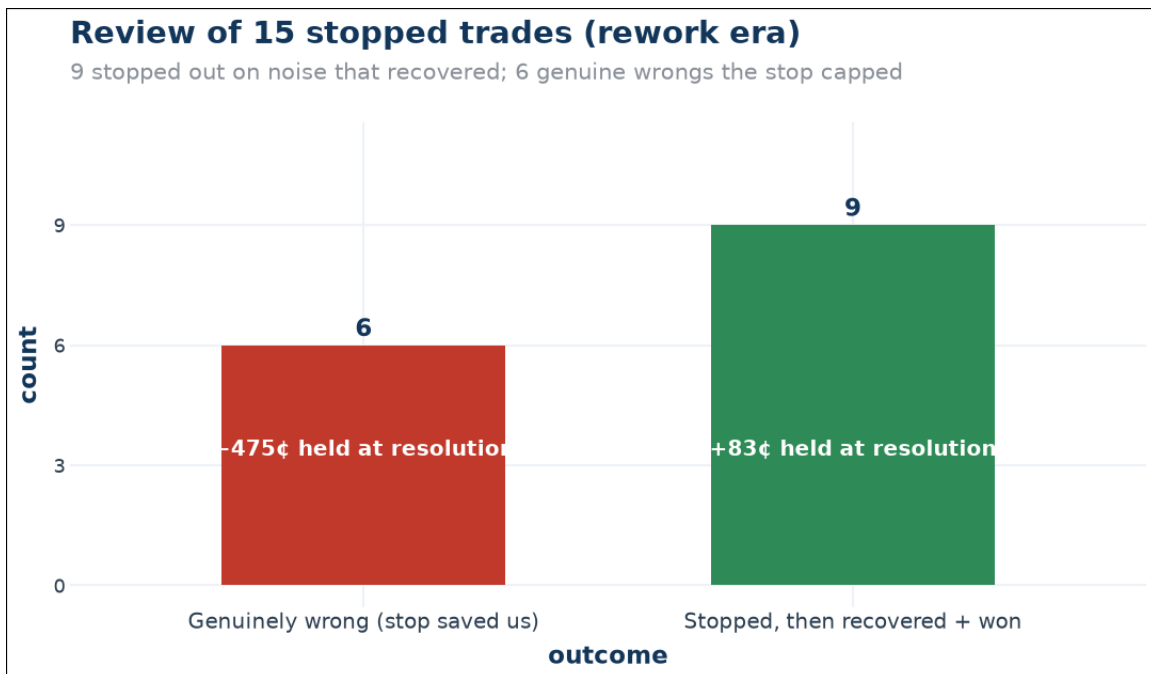
The standard hour, trade by trade: gold NO at 90c banked 6.74c. Silver YES at 81c held to the bell for 23c. Gold NO at 93c banked 2.74c. Silver NO at 93c held for 8c. Gold YES at 87c banked 4.74c. Silver YES at 86c held for 16c. Gold NO at 93c banked 4.74c. Silver NO at 81c held for 23c. No heroics. The bot let the market do the work and got out of its way.

The first evening of the standard config: 21 trades, 20 wins, 1 loss – and the one loss was capped at 6.74c at the stop level. Live prediction accuracy: 89.5% model, 99% market.

And the account what-if from real fills: trades that agreed with the model won 92.5%; trades that went against it won 6.4%. That gap is the whole reason the model gets a vote.



Cumulative paper P&L; across those hours



Why the stops changed: nine recovered and won, six were genuine wrongs

Then came the part I did not want. When I made the backtest honest – when it used the live engine's own functions and real candle books instead of a friendly approximation – the full history said the current config lost \$76. I could have argued with it. Instead I cut the lanes it said were bleeding, then swept the hatch it said was too wide, and the same history flipped to +\$6.29.

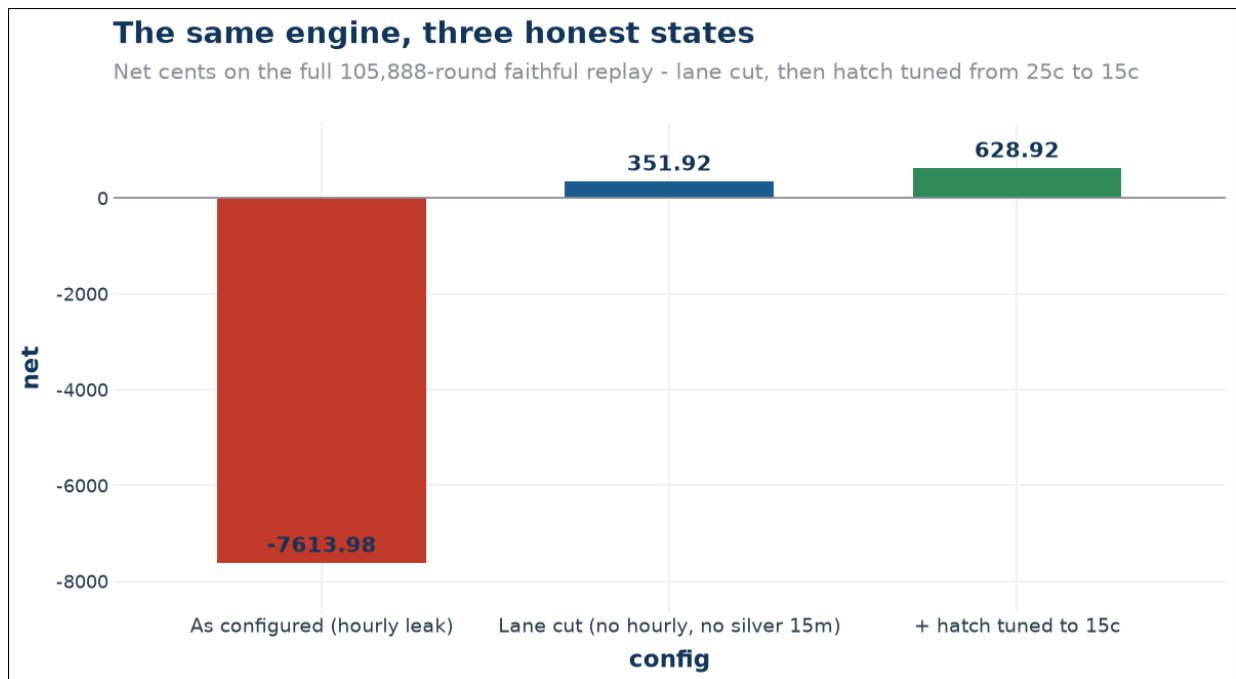
7. We're not done – the engine as it stands

This is the new chapter, written after the honest backtest. If you read nothing else, read this one – it is the current state, the current numbers, and the current plan, all in one place.

The honest backtest is not a simulation of my rules. It is my rules. It imports the live engine's actual decision functions, mirrors the GBM gate and the exit loop line for line, uses the watcher's real config (the enabled lanes, the 1.5–8 minute snipe window, the 0.63c fee, the normal risk governor), and replays every round minute-by-minute against real Kalshi candle books – scoring the same models from only the candles that existed at each scan minute. No lookahead. Then it holds the result up next to the live engine and dares them to disagree.

What it found, on all 105,888 rounds, in three honest states:

State	Trades	Win rate	Net
As configured (hourly leak)	2,823	73.6%	-\$76.14
Lane cut (no hourly, no silver 15m)	608	78.1%	+\$3.52
+ hatch tuned to 15c (shipped)	608	77.3%	+\$6.29

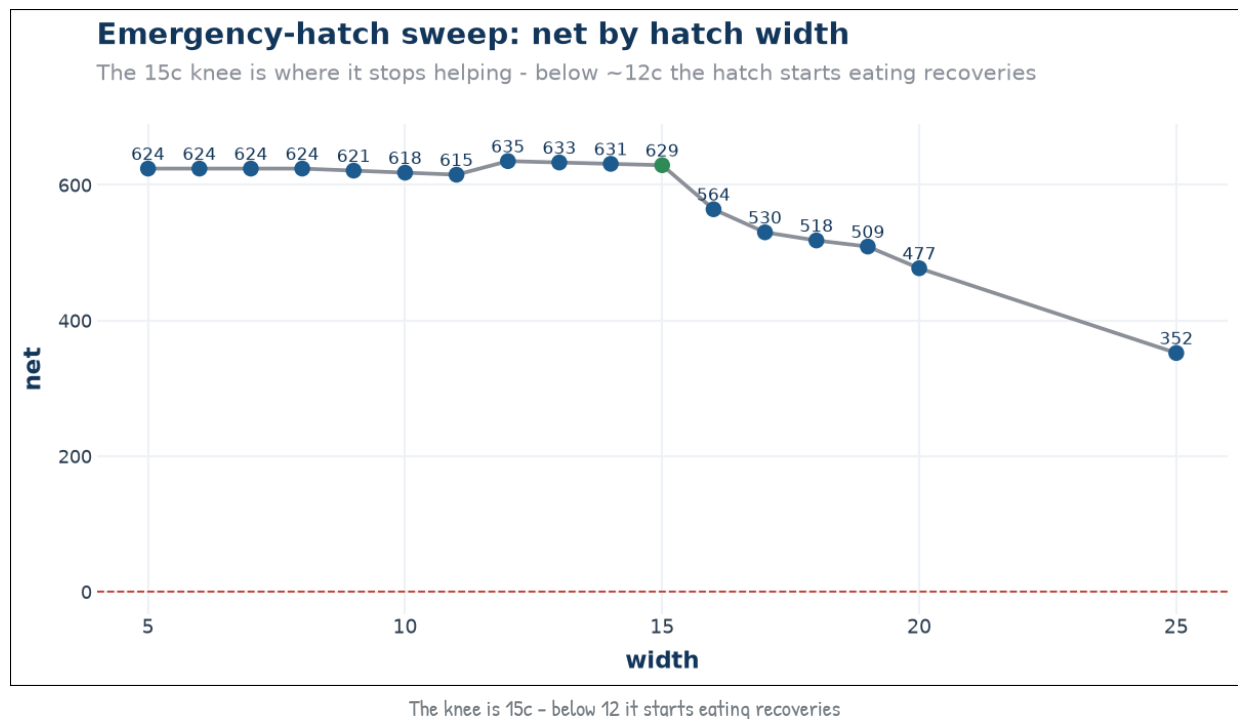


The same engine, three honest states – lane cut, then hatch tuned

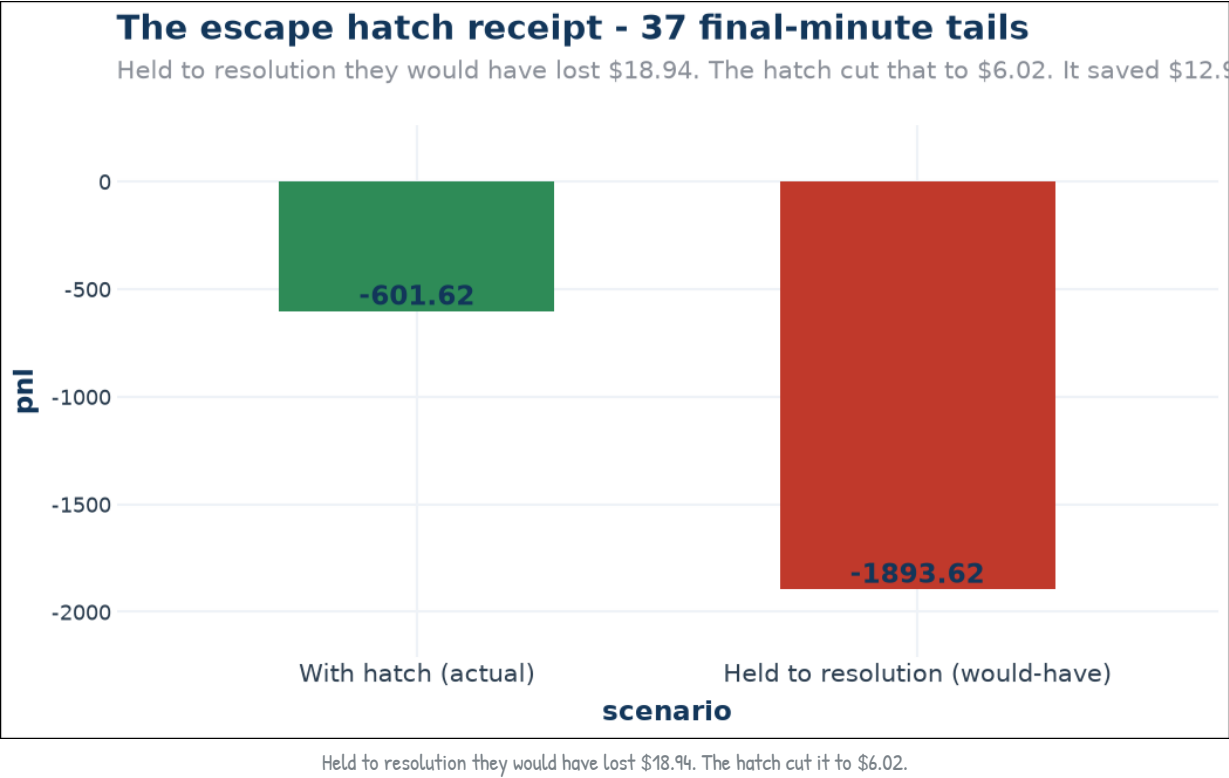
The first row is the confession: the system I had running was losing money, and most of it – \$75 of the \$76 – came from hourly gold and silver, the lanes I was most excited about. The second row is the lane cut: freeze the hourly markets, drop the slightly-negative silver 15m lane. The third row is the hatch tune: sweep the final-minute emergency exit from 25 cents down to 15, where the curve said the knee was.

The hatch sweep, width by width, on the full history:

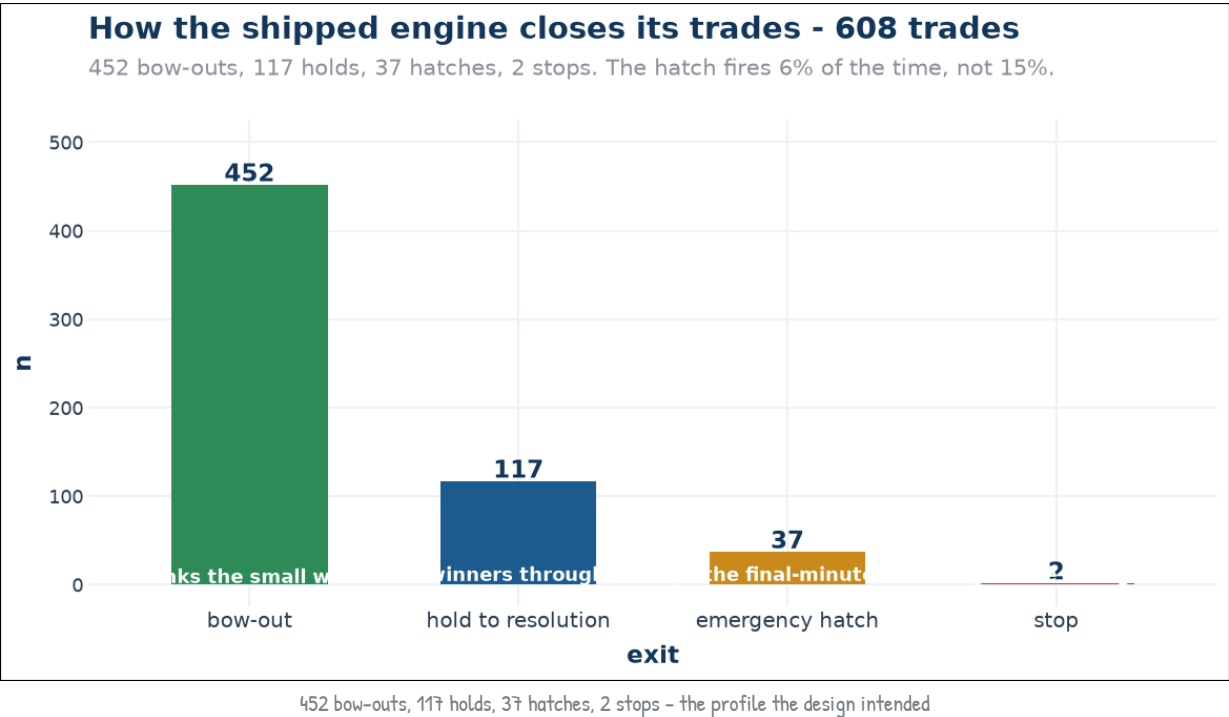
Width	Net	Width	Net
25	+\$3.52	14	+\$6.31
20	+\$4.77	13	+\$6.33
19	+\$5.09	12	+\$6.35
18	+\$5.18	11	+\$6.15
17	+\$5.30	10	+\$6.18
16	+\$5.64	5	+\$6.24
15	+\$6.29		



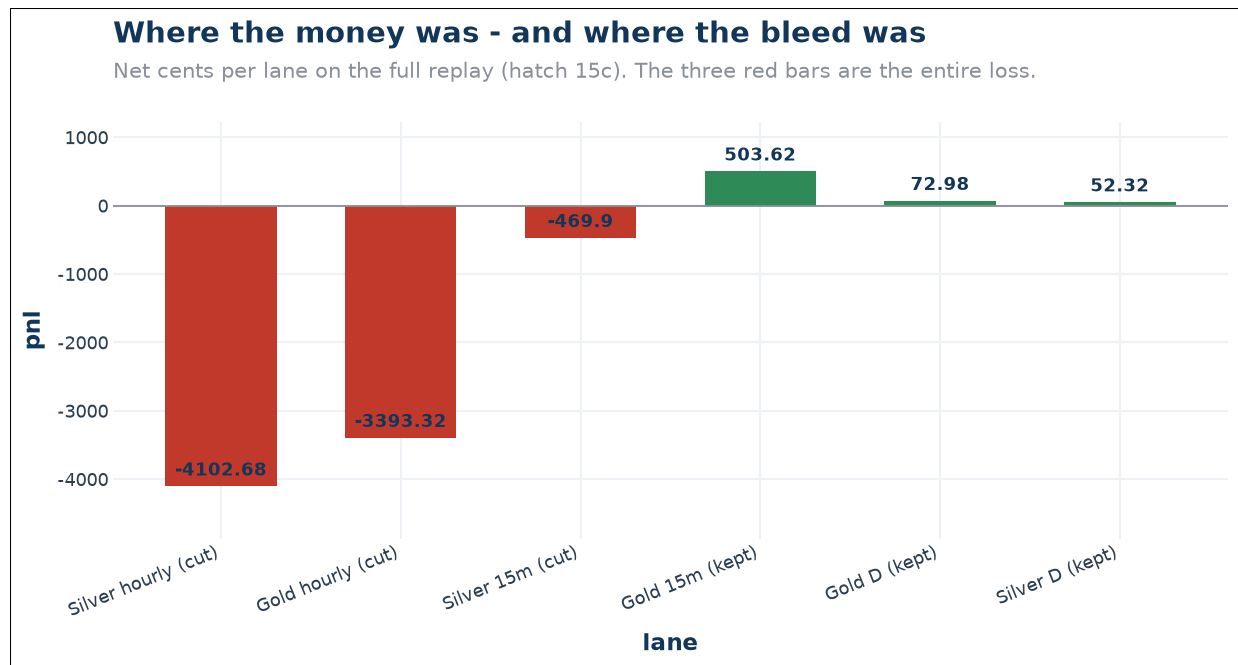
Why the hatch matters so much: in the old config it fired 420 times at -26c each, and that single line - -\$110 - was the biggest loss in the whole system. At 15c on the shipped lanes it fires 37 times, and it pays for itself. The receipt:



The exit mix of the shipped engine, and where every dollar came from:



Per lane, the verdict is clean. The three red bars are the entire loss; the green bars are the engine that remains:



The lane cut was not a guess - the replay measured it

The shipped numbers, full dataset: 608 trades, 77.3% win rate, +\$6.29 net, +1.03c average per trade, \$1.96 max drawdown. On paper, at \$1 stakes. The caveats that keep me honest: the replay samples candles once a minute while live scans every few seconds, fills at candle close instead of real-time, and uses flat \$1 sizing - so the real number will differ a little, in either direction. The point of the exercise was never the exact cents. It was knowing, for the first time, that the engine is not the reason I am not there yet.

What is shipped right now, as of this writing: hourly gold and silver are frozen in the engine itself (the freeze list existed but the universal scanner was ignoring it - that was the leak), silver 15m is excluded from the universal lane, the hatch is 15 cents, and the watcher is running on paper with all of it. The dashboard plays the full history back and shows these numbers live. The next lever is the crypto 15m lane, which currently never fires under the GBM agreement gate - it needs its own tuning pass through this same replay before it touches paper.

The honest scoreboard: -\$76 (as configured) to +\$3.52 (lane cut) to +\$6.29 (hatch tuned). Same engine, same history, same code - different decisions about what to trade and when to leave.

8. Risk – the stop policy, and the hatch I had to tune

A stop is a promise you make to yourself before the trade: if this goes wrong, it costs me this much and not a cent more. My old bot broke that promise in both directions – it fired on noise, and when a real break happened it filled at whatever the market panicked to. The new policy fixes both.

The evidence base is the rework's fifteen stops: nine fired on transient dips that recovered and won (the flat stop cost me 188c of forgone wins), and six were genuine wrongs where the stop saved 344c. Net, stopping still saved money – the problem was never stopping, it was stopping on noise and filling at crash prices.

Control	What it does
Tiered stops	entry $\geq 90c$ gets 15c of room; below that, 8c - high prices mean the dip is almost always noise
Recovery grace	a dip has 3 minutes to prove it's real; if the market comes back, the winner stays
Limit-stop fills	the exit is marked at the stop level - a crash price is never booked
Fast checks	every 1.5 seconds, paper and live
Entry timing	no mid-price entry ($< 95c$) with under 4 minutes to close - a dip needs room to recover
Final-minute hold	GBM-confirmed entries in the last 2 minutes ride to resolution
Emergency hatch	15c quit in the final minute - tuned by sweeping 25 down to 5 and reading the curve
Hard bans	no martingale, no revenge trading, no leverage, no both-sides arbs
Real money	off until the paper record earns it

The two newest rules came from the same loss. A silver trade entered 3.6 minutes before the bell dipped and was stopped 52 seconds before close – the closing-soon rule fired, and no grace length could help. The fixes: don't take that entry at all (timing guard), and for the trades I do take in the final minutes, hold them through the noise – with a tuned emergency quit in the last minute so a true collapse still gets out before a full loss.

Here is the honest answer to the question I kept asking myself: how can the escape hatch be the biggest loss line if it is supposed to escape before the dip? Because it is escaping – that is why each one costs 15 cents instead of 90. The damage is the capped price of being wrong that often, and the fix was not a better hatch, it was better entries. The lane cut dropped the hatch from 420 uses to 37. The hatch was never the leak. The lanes were.

9. The economics – one bot, a fleet, a thousand bots

The unit of edge is a percent, not a cent. Buy at 90 cents, it settles at 100: +11.1% on what you deployed. The cents in my reports are the per-contract unit; the dollars scale with position size – and position size scales with the balance you are willing to back it with.

Why growth accelerates: the return is a percentage of your stake, and your stake is a percentage of your balance. At \$200 balance and a \$50 entry, ten perfect trades are +\$55.60 – the resolution stops making cents and starts making dollars. The ladder:

Balance	Stake (25%)	One win (+11%)	One stop
\$5	\$1.25	+\$0.14	-\$0.11
\$30	\$7.50	+\$0.83	-\$0.67
\$100	\$25	+\$2.78	-\$2.22
\$200	\$50	+\$5.56	-\$4.44
\$500	\$125	+\$13.89	-\$11.11

(pencil) I had a \$200-to-\$500-in-month-two table in an earlier draft. It was wrong – that is compounding applied to the wrong number. This one is percent-of-position, and it is the one I ship.

Now the network math. Assume the current strategy, a 95.2% win rate (one miss in every 21 trades), and every bot eventually trading \$20 stakes from a \$200 balance. A win pays \$2.22; a capped stop costs \$1.78; a full final-minute loss costs \$20. Per trade, the expected net is about +\$2.03 with capped stops, or +\$1.16 if the rare miss is a full loss.

Bots	Trades/day each	Per-bot / month	Network / month
1	30	\$1,829	\$1,829
1	100	\$6,095	\$6,095
10	100	\$6,095	\$60,950
100	100	\$6,095	\$609,500
1000	100	\$6,095	\$6,095,000

The full economics at a glance

How to read this table: every row assumes \$200 banked as backing. The stake is a percentage of that balance. The pace is trades per hour at three levels – Optimal is what the engine could reach across many markets, Accepted is what today's gate actually lets through, and Expected is the realistic sustained pace. Per-trade return is net of

fees and assumes the measured model: a 95.2% win rate, +11.1% on wins, -8.9% on capped stops, and the 0.63c-per-contract entry fee (1.26c round-trip).

Read the highlighted row: \$200 balance, \$20 stakes (10%), 5 trades/hour for 16 hours = 80 trades a day, each netting \$2.03 after fees. That is \$162 a day, \$4,872 a month – with about 4 expected losses a day, each capped at \$1.78.

Stake	Pace	Trades/day	Return/trade	Fees/trade	Day (16h)	Month (30d)	Losses/day
5% (\$10)	Optimal 12	192	\$1.02 / 10.2%	\$0.06	\$195	\$5,846	~9
5% (\$10)	Accepted 5	80	\$1.02 / 10.2%	\$0.06	\$81	\$2,436	~4
5% (\$10)	Expected 6	96	\$1.02 / 10.2%	\$0.06	\$97	\$2,923	~5
10% (\$20)	Optimal 12	192	\$2.03 / 10.2%	\$0.13	\$390	\$11,693	~9
10% (\$20)	Accepted 5	80	\$2.03 / 10.2%	\$0.13	\$162	\$4,872	~4
10% (\$20)	Expected 6	96	\$2.03 / 10.2%	\$0.13	\$195	\$5,846	~5
25% (\$50)	Optimal 12	192	\$5.08 / 10.2%	\$0.32	\$974	\$29,232	~9
25% (\$50)	Accepted 5	80	\$5.08 / 10.2%	\$0.32	\$406	\$12,180	~4
25% (\$50)	Expected 6	96	\$5.08 / 10.2%	\$0.32	\$487	\$14,616	~5

The 25% rows are aggressive, not promised: at \$50 stakes a single wrong final-minute hold costs \$50, and one of those per 21 trades cuts per-trade net roughly in half. The gate exists precisely so the accepted pace stays at the safe end of this table.

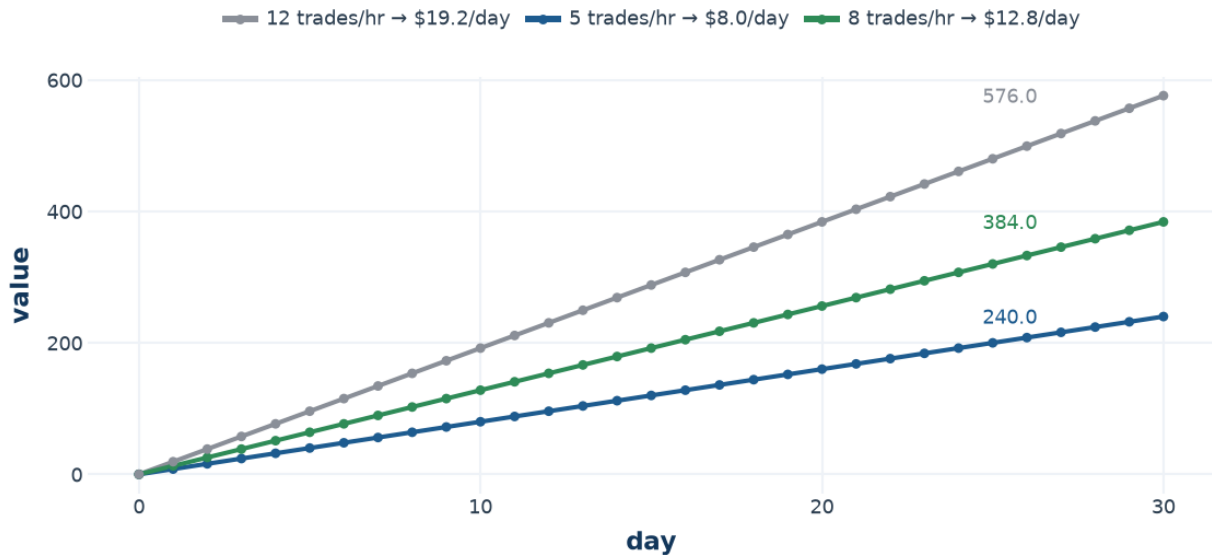
A family of four can live on about \$1,800 a month – one bot at 30 trades a day covers that. A college student needs about 10 trades a day. Ten friends running bots together: hobby money on autopilot. A hundred: a quit-your-job number. A thousand: the dream, with the caveats printed right next to it.

Two things cap it before the math does:

- Liquidity. A 15-minute round has maybe \$50-100k of depth. A thousand bots can't all trade gold at once – the fleet must spread across hundreds of markets, which is why adoption and market coverage are the real requirements.
- The loss tail. At \$20 stakes, a full final-minute loss costs \$20, and one of those per 21 trades drags the per-trade net from \$2.03 to \$1.16. The win rate is the second-most-important number in the whole system, after the stop.

Examples of returns — 10¢ per trade, 16 hrs/day

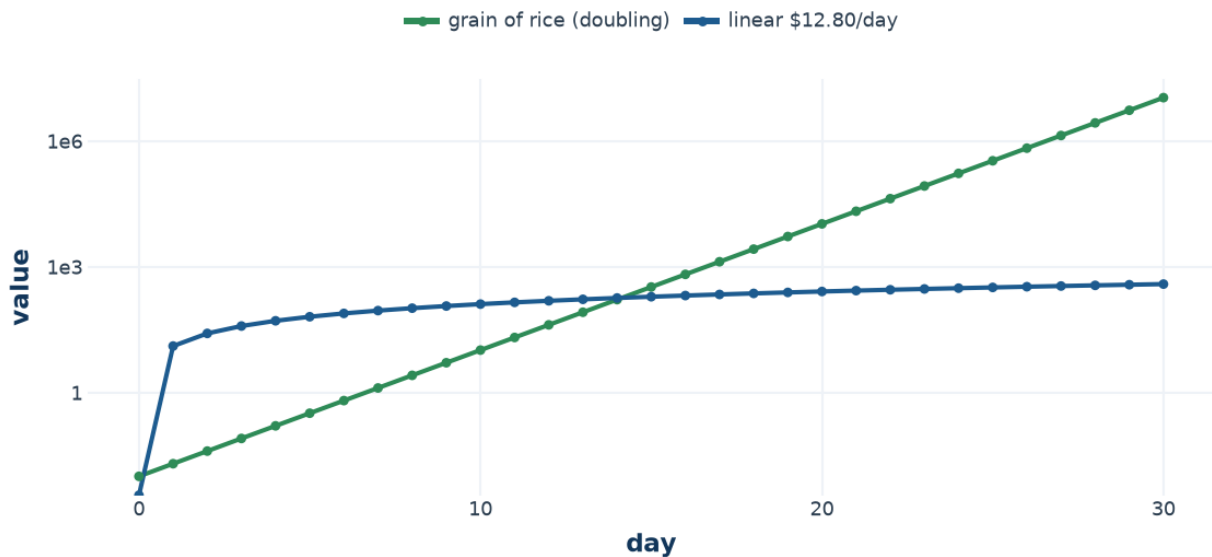
Paper scenario math: 5 / 8 / 12 trades per hour over 30 days



Flat per-trade scenario math – before position sizing

Why compounding beats linear — the grain-of-rice curve

Doubling a grain of rice vs a fixed \$12.80/day (log scale)



Why compounding beats linear — the grain-of-rice curve

10. The pool – how a fleet works

One bot can watch a handful of markets every night. Ten bots can watch all of them, and each one gets to be lazier because the others cover the rest. The pool is the nervous system that lets workers share what they find instead of each learning everything alone.

The players. The main node owns the strategy configs and is the only one that pushes verified strategies down. Workers run the same engine on their own slice. A small Cloudflare Worker in front of a KV store is the shared inbox: signals, ledgers, strategy versions.

The credits economy. Publishing a good signal costs a few credits in escrow. Another worker who verifies it gets paid for the verification, and the original poster is refunded when the signal proves actionable – so posting good information pays for itself, and posting garbage costs. A single piece of information can only be refunded once. Workers cap at 200 credits; anything above 100 flows into the pool's float, which is used to verify incoming information, so the network stays self-balancing.

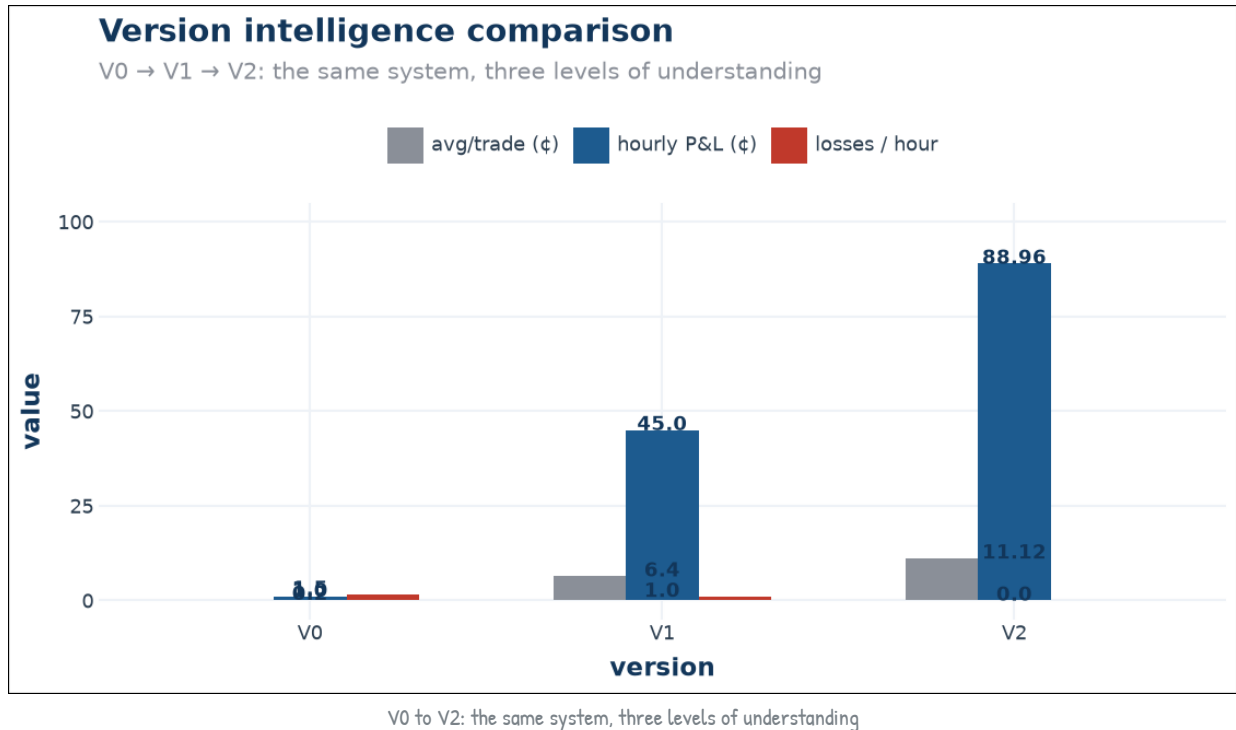
The guardrails. Any node can publish at most one post per 10 minutes; any node can verify at any time; only the main node pushes strategy versions. Workers specialize naturally – a node in another timezone covers the overnight markets, a commodity-only node runs just those lanes – and because everyone shares for a flat, small rate, the fleet converges toward the same knowledge over time. That's the daisy chain: one worker's discovery becomes everyone's.

Current status, honestly. The pool is disabled during the paper test. The live worker burned its daily KV quota through sync fan-out, not trading – the /feed endpoint did a KV list and a read fan-out on every call, and one idle node polling every 60 seconds through three overlapping paths used up the free tier in under a day. Fixed client-side (5-minute polls, dedupe); a worker cache is staged and needs deploying from the account that owns the pool. The roadmap: deploy the cache, re-enable, run two nodes on paper side by side, then scale positions within each market's liquidity – not beyond it.

And here is the part that is also a referral: when you fund a new account, the app asks you for a code before you fund it – because the pool works on introductions. Someone who reads this far has a code. Use it. The daisy chain only grows if the information actually travels.

11. Where I'm going next

My bot didn't get smarter by adding parameters. It got smarter by learning from its own outcomes, refusing weak setups, and finally understanding the difference between noise and signal. That's not magic - it's a trader who started keeping a journal.



What's next is boring in the best way: a crypto 15m lane tuned through the same honest replay until it clears the GBM agreement gate, more markets with trained models (16 series are in the dataset), day after day of net-positive paper hours, then the sizing ladder - \$1 paper stakes today, \$5, \$20, \$50 as the record earns it. Then the fleet: two nodes on paper side by side, sharing signals, then a real network of workers in different markets and timezones.

One perfect hour isn't a track record. I know that. Neither is one honest backtest that finally comes out positive. But for the first time since I started, the bot isn't the reason I'm not there yet - and that is a strange, good feeling at 2 AM.

12. Straight answers

What is the actual edge?

Ultra-short markets converge: a side implied at 90%+ settles that way ~96-99% of the time. I enter late, with a confident model and an observer's agreement, ride winners through noise, and cap the rare wrong call at a fixed stop level - 15 cents in the final minute.

Why did the old system fail?

It was a static formula with no sense of context, stops that gaped 17-65c on paper, and no journal. The rework fixed all three, and every fix maps to a measured loss.

Why is the honest backtest net negative for the old config?

Because 1,650 of 2,823 trades were hourly gold and silver - lanes that resolve the wrong way 27-36% of the time despite being priced as near-certain. Cut the lanes, tune the hatch, and the same history is positive.

Can it really make money?

On the full 105,888-round faithful replay, the shipped config nets +\$6.29 at \$1 stakes - thin, real, and positive. The dollars come from the sizing ladder once the paper record earns it, not from pretending the edge is bigger than it is.

What is the LLM for?

It's the observer - it reads each round like a trader would and gives a one-line opinion. Advisory only, logged, scored, never in charge.

Why a three-minute grace on stops?

Because nine of fifteen stops in the old system fired on dips that recovered and won. A dip gets time to prove it's real; a break that stays still closes at the stop level.

Why 15 cents for the hatch?

I swept the emergency quit from 25 down to 5 on the full history. The curve rises to a knee at 15 and flattens; below 12 it starts eating recoveries. It is the width that lets the final-minute hold survive.

What are realistic returns?

Paper standard hour: 8W/0L, +88.96c, 11.12c average, \$0 drawdown. Full-history shipped config: 608 trades, +1.03c average. At \$20 stakes with a 95.2% win rate, one bot projects ~\$60-200 a day depending on how many trades the market offers.

When can this go live?

After day after day of net-positive paper hours, a passing leakage audit, and confirmation that stops fill at the stop level. Then the sizing ladder begins at \$1.

What are the biggest risks?

A real reversal on a high-price entry (capped at the stop), a regime change that breaks the pattern until the journal catches it, Kalshi fee or availability changes, liquidity limits at scale - and one honest backtest not being a track record.

Does it use leverage?

No. No martingale, no revenge trading, no leverage, no both-sides. Real money is off until the paper record earns it.

Why run it at all?

A few watts for a self-correcting system with a quantified edge and capped downside - and the first worker in a fleet that makes quiet days into free gains. Also, I want to make pancakes on a Tuesday.

Disclaimer: Past performance is not a guarantee of future results. Trading involves real risk. Kalshi is not available in all states. Nothing in this document is financial advice. All figures are read live from the system state and Kalshi settlement records on 2026-08-20 07:58 UTC.

Companion markdown: docs/00-thesis.md, docs/06-performance.md, docs/13-versions.md, docs/14-reasoning.md, docs/15-returns.md, docs/18-pool.md, ARCHITECTURE.md. Charts: docs/charts/*.png (plotnine).

One last thing

I told you this was an explanation of my madness. Here it is, plainly. I do not want to trade my hours for money until I am too tired to enjoy the money. I want a fast, stable, monetary future for my family, and I want it to keep working while I am making pancakes. No one will ever pay me more than me, because no one else is this motivated. And everyone – everyone – should be able to just exist.

The rice is real. One grain that doubles every day looks like nothing for almost a month, and then it is a fortune. This system is the first option: it looks like nothing for a while – cents, paper, one perfect hour – and then, if the record earns it, it stops looking like nothing. I have been wrong about a lot of the details. I have been right about the direction, and the direction is what I am asking you to consider.

Callbacks, because every good closing has them: the 92-cent question, the grain of rice, the escape hatch that was never the leak, the lanes that were, the config I touched five times in one day, the 86 restarts, the 2 AM reviews, the \$76 the honest backtest made me look at, and the \$6.29 it let me keep. If you made it this far, you are the kind of person who reads the whole manual before assembling the furniture – and that is exactly the kind of person who should be running one of these.

If you're smart, and you read all of this – yours. When you fund your account, use the referral code you were given. It is how the daisy chain grows. One worker's discovery becomes everyone's, and the network only works if the information actually travels.

I am not asking you to trust the dream. I am asking you to check the receipts – every number in this document is reproducible from the code and the dataset, and the honest backtest is a button on the dashboard. Run it. Argue with it. It argued with me, and I am better for it.

~~Please let this work. Please let this work.~~

(pencil) It already does, on paper. Now prove it live – one net-positive day at a time. Then go make pancakes.

– the guy who built it, 2 AM, somewhere between restart 86 and the next one